

Distributed Systems: Project VASH

VASH File server

Our file server serves the entire file content on Open (whole-file caching).

It uses a generation based cache validation.

Each file has a generation number that starts at 0 and increments on every modification. Clients store the generation number when they fetch a file, and can later use `TestAuth` to check if their cached copy is still current (if it has the same generation number as the server's current version).

Clients know the servers and will contact them in a given order, each server having a different priority. If a server doesn't answer a client request, they will timeout and contact the next server on the list. That's made possible by the fact that our servers share the same metadata. These metadata are also persistent on the disk in order to make it possible for a server to recover fast and reliably.

Our server also supports **atomic writes** by using a temp file and rename on Close.

Finally, we also made sure to use mutex everywhere to ensure that two concurrent requests would not corrupt our data.

Fault Tolerance

Our file server can also handle node failures.

Handling a node crash and recovery:

Each node maintains an operation log with the last operations. Every operation gets a unique ID that increases over time. When a client sends a request to any node, that node broadcasts the operation to all other nodes. This keeps our nodes in sync.

We added a flag called `IsReplication` to distinguish between client requests and peer broadcasts. When a node receives an operation from a client, it broadcasts it to peers. But when it receives an operation from another node, it just stores it locally without broadcasting again. This prevents infinite loops.

All operations are now broadcasted, including Open. We do this because Open changes the metadata, and all nodes need to absolutely keep their metadata in sync. If we didn't broadcast it, nodes would have inconsistent generation numbers.

Each node saves two things to disk: the metadata and the operation log. We persist these after every modification. The metadata goes into `.metadata.json` and operations go into `.operations.json`. Both files live in the output directory.

Handling a client crash during write:

Our clients do whole-file caching, so all the modifications happen at first on a local copy of the file. The writes are communicated to the server when the client closes the file, so a crash during write cannot modify the data on the servers. The server also does an atomic write, using a temp file.

Recovery process:

When a node crashes and comes back up, it follows these steps:

1. Load the saved metadata from disk
2. Load the saved operations from disk
3. Contact all peer nodes listed in `serverAddrs`
4. Ask each peer for operations newer than its last known operation ID
5. Replay the missing operations in order (only replay each operation once)
6. Resume normal operation

The recovery is completely automatic. As long as at least one peer is online, the recovering node can catch up.

Known limitations (by design):

We only keep 100 operations in memory. If a node is down for too long and misses more than 100 operations, it won't be able to fully recover. We could improve that by saving the logs to the disk.

VASH Primality Checker

As the primality checker algorithm we use the built in `isProbablyPrime()` which is deterministic for 64 bit integers. We are using an asynchronous Coordinator Worker design. The workers ask the coordinator for an assignment and the coordinator simply directs them to the file server along with the assignment details. We have support for 2 design paradigms, depending on use case

1. Offset chunking, Where the coordinator simply gives offsets to the workers and the workers all work in the large file, but process only a subset of the file. This should be used in a datacenter setting where network throughput is high and workers share the same cache. In such a setting it is faster than Physical file chunking because we avoid repeated file IO. Because of the Andrew File system design, we would need to send the whole file even if the worker only processes a small portion of it. To enable it in `cmd/coordinator` [main.go](#), the `c.chunkFiles` function should be called with the `numLines` to 0 and `ChunkSize` to the desired size in bytes, also, in the `cmd/worker` `main.go` call the `primeChunks` function with parameter `true`. For the physical file do the opposite in both cases.
2. Physical file chunking, where the coordinator creates separate files which dramatically lowers the network load. Each worker only gets as much as it is going to process. This is slow, however, because there is a lot of file IO involved. On the other hand the RAM, cache storage and Network requirements are much lower. This is ideal for a truly distributed system where the devices are located in different locations and are bound by real world network latency and throughput. In a normal setting fast storage is much cheaper than fast networking.

With both paradigms, each chunk is processed, it is sorted then it is outputted to a file in the file system. This makes the design highly asynchronous and very easy to recover after a failure.

Merging and Deduplication

We use a 2 stage distributed merging system.

1. In the first stage each worker utilizes AFS cache locality to quickly and efficiently merge its own files that it has in the cache.
2. In the second stage, the coordinator retrieves each worker's merged file and does a final deduplicated merging. In both stages we use a K-way merging algorithm which utilizes min-heaps. It is highly inspired by Linux's `sort -u -n` command. Each worker sorts the chunk output which makes it very easy to open all outputs in parallel and merge them with very low memory utilization. We made the merge buffer size customizable depending on worker RAM capabilities. Currently the number of file handles is limited by the OS to 1024 but this could be modified.

Snapshot and Fault recovery

The coordinator initiates the snapshot recording in a timely interval since our cases are rather small; it is much smaller than it would be in a real world scenario. It does this by sending marker messages to all the workers that are registered to it. The workers then call the Coordinator RPC function and send their state along with any other recorded messages. And also the in-transit messages are saved as well and can be resent when the recovery happens. In the global state we then store everything we need to recover at the same stage as it was before the crash. Including the local state of the coordinator and that of all workers. The snapshot is saved in a json file. Which will be rewritten after each new snapshot.

After a manual reboot, both the coordinator and the workers first check if a snapshot exists, if it does when they are constructed, they get the data from the json file and skip the normal initialization.

Where all necessary fields are reset to the value before the snapshot has been taken.

For Fault recovery we were also thinking of implementing a heartbeat and monitoring of workers. Where we continuously send heartbeats from the worker to the coordinator. And if we haven't got a heartbeat for longer periods of time it would reassign the task. Since if a worker dies and does not come back to life, it will hinder the progress. We have commented on this part since it's outside of the scope of this project.